King Fahd University of Petroleum & Minerals – Systems & Control Engineering

Assessment 2 Technical Presentation: Autonomous Trajectory & Hardware Commands

Course: SCE 672 – Advanced Robotics (Prof. Aamer Iqbal Bhatti) **Date:** October 2026
Student: Raian E Rabbani (ID: 202617880) **Target:** AprilTag T_3 ($d_0 = 0.832$ m, Standoff: 0.20 m)
Repository: https://github.com/idiot28/sce672-assessment2

1. 2D Map of the Last Successful Run

Figure 1 shows the executed ground trajectory against the planned differential flatness trajectory from the latest successful experimental run (`run_20261028_170850_tag3`). The robot initially sweeps in `SEARCHING` mode, locks onto Target AprilTag T_3 at relative distance $d_0 = 0.832$ m, aligns its heading face-to-face ($\Delta\theta = 180°$ anti-parallel to the tag surface normal), and executes the smooth cubic flatness trajectory. Nonholonomic lateral velocity is strictly zero ($v_y \equiv 0.000$ m/s). The robot decelerates smoothly and arrests motion at the 20 cm standoff boundary, reaching a final standoff distance of **0.2038** m (**0.38** cm error).

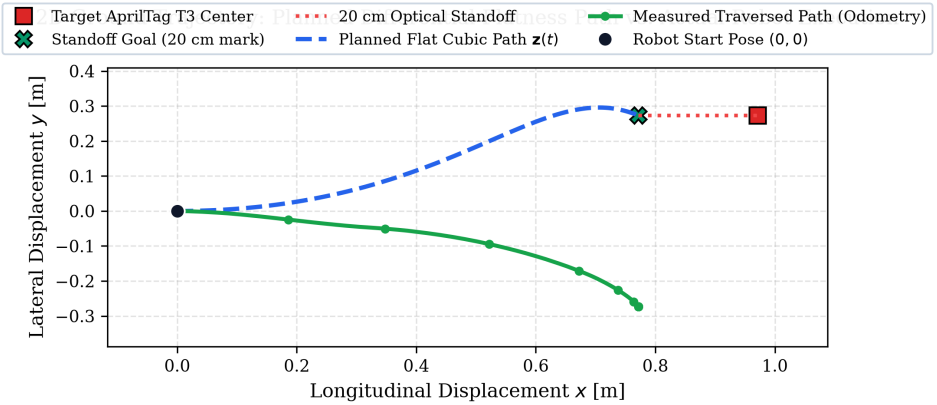


Figure 1: 2D Ground Path Map: Planned differential flatness trajectory (blue dashed line) vs. actual measured robot path (green solid line), terminating squarely at the 20 cm standoff boundary in front of AprilTag T_3.

2. The Generated Cubic Polynomial Formula & Flatness Solver

Under differential flatness for unicycle kinematics ($\dot{x} = v\cos\theta, \dot{y} = v\sin\theta, \dot{\theta} = \omega$), the 2D Cartesian position $\mathbf{z}(t) = [x(t), y(t)]^T$ is a flat output. Each flat coordinate is parameterized as a smooth C^2 cubic polynomial over mission execution horizon $T = 2.600$ s:

$$z_j(t) = \alpha_0^{[j]} + \alpha_1^{[j]} t + \alpha_2^{[j]} t^2 + \alpha_3^{[j]} t^3, \qquad j \in \{x, y\} \tag{1}$$

The boundary conditions for Tag T_3 (initial crawl start $\dot{x}_0 = 0.080$ m/s, final standoff point $x_f = 0.1152$ m, $y_f = -0.0811$ m, terminal halt $\dot{x}_f = 0.0288$ m/s, $\dot{y}_f = 0.0409$ m/s) define the linear boundary value system $\mathbf{M}(T)\boldsymbol{\alpha} = \mathbf{b}$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 2.6 & 6.76 & 17.576 \\ 0 & 1 & 5.2 & 20.28 \end{bmatrix} \begin{bmatrix} \alpha_0^{[x]} \\ \alpha_1^{[x]} \\ \alpha_2^{[x]} \\ \alpha_3^{[x]} \end{bmatrix} = \begin{bmatrix} 0.0000 \\ 0.0800 \\ 0.1152 \\ 0.0288 \end{bmatrix}, \qquad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 2.6 & 6.76 & 17.576 \\ 0 & 1 & 5.2 & 20.28 \end{bmatrix} \begin{bmatrix} \alpha_0^{[y]} \\ \alpha_1^{[y]} \\ \alpha_2^{[y]} \\ \alpha_3^{[y]} \end{bmatrix} = \begin{bmatrix} 0.0000 \\ 0.0000 \\ -0.0811 \\ 0.0409 \end{bmatrix} \tag{2}$$

Inverting $\mathbf{M}(2.60)$ yields the solved polynomial coefficient vectors $\boldsymbol{\alpha}_x = [0.00000, 0.08000, -0.02147, 0.00298]^T$ and $\boldsymbol{\alpha}_y = [0.00000, 0.00000, -0.05173, 0.01528]^T$. The resulting closed-form analytical equations generated and executed in code (`src/flatness_math.py`) are:

$$\boxed{x_{\text{ref}}(t) = 0.080000\,t - 0.021470\,t^2 + 0.002979\,t^3 \quad \text{(m)}} \tag{3}$$

$$\boxed{y_{\text{ref}}(t) = -0.051727\,t^2 + 0.015280\,t^3 \quad \text{(m)}} \tag{4}$$

$$\boxed{v_{\text{ref}}(t) = \sqrt{\dot{x}_{\text{ref}}(t)^2 + \dot{y}_{\text{ref}}(t)^2}, \quad \omega_{\text{ref}}(t) = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2}, \quad v_y(t) \equiv 0.000 \text{ m/s}} \tag{5}$$

2.1 Algorithm Implementation in Code: Matrix Boundary Solver

The following listing illustrates the boundary value solver from `src/flatness_math.py`. It constructs the 4×4 linear system $\mathbf{M}(T)\boldsymbol{\alpha} = \mathbf{b}$ for position coordinates and inverts it to produce the analytical coefficients $\boldsymbol{\alpha}_x$ and $\boldsymbol{\alpha}_y$:

```python
def solve_flat_polynomial(z0: float, zdot0: float, zf: float, zdotf: float, T: float) -> np.ndarray:
    """Solves M(T) * alpha = b for a single Cartesian coordinate:
       z(t) = alpha_0 + alpha_1*t + alpha_2*t^2 + alpha_3*t^3"""
    T = max(float(T), 0.1)
    M = np.array([
        [1.0, 0.0, 0.0,     0.0],
        [0.0, 1.0, 0.0,     0.0],
        [1.0, T,   T**2,    T**3],
        [0.0, 1.0, 2.0 * T, 3.0 * (T**2)],
    ], dtype=np.float64)
    b = np.array([z0, zdot0, zf, zdotf], dtype=np.float64)
    return np.linalg.solve(M, b)

def compute_boundary_matrix_inv(T: float) -> np.ndarray:
    """Returns exact analytical 4x4 inverse boundary matrix M(T)^(-1)."""
    T2 = T * T
    T3 = T2 * T
    return np.array([
        [1.0,         0.0,        0.0,         0.0],
        [0.0,         1.0,        0.0,         0.0],
        [-3.0 / T2, -2.0 / T,    3.0 / T2,   -1.0 / T],
        [ 2.0 / T3,  1.0 / T2, -2.0 / T3,    1.0 / T2],
    ], dtype=np.float64)
```

Listing 1: Boundary value problem solver for cubic polynomial flatness trajectory (`src/flatness_math.py`)

2.2 Algorithm Implementation in Code: Differential Flatness Output Inversion

At each 20 Hz control tick during the `CHASING` state, the node evaluates the flat output coordinates and algebraically recovers the feedforward linear speed $v(t)$ and angular turn rate $\omega(t)$:

```python
def evaluate_flatness(alpha_x: np.ndarray, alpha_y: np.ndarray, t: float, T: float):
    """Evaluates 2D flat cubic curve z(t) and recovers unicycle control inputs:
       v(t)     = sqrt(xdot^2 + ydot^2)
       omega(t) = (xdot*yddot - ydot*xddot) / (xdot^2 + ydot^2)"""
    tau = float(np.clip(t, 0.0, T))
    tau2 = tau * tau

    x = alpha_x[0] + alpha_x[1]*tau + alpha_x[2]*tau2 + alpha_x[3]*(tau2*tau)
    xdot = alpha_x[1] + 2.0*alpha_x[2]*tau + 3.0*alpha_x[3]*tau2
    xddot = 2.0*alpha_x[2] + 6.0*alpha_x[3]*tau

    y = alpha_y[0] + alpha_y[1]*tau + alpha_y[2]*tau2 + alpha_y[3]*(tau2*tau)
    ydot = alpha_y[1] + 2.0*alpha_y[2]*tau + 3.0*alpha_y[3]*tau2
    yddot = 2.0*alpha_y[2] + 6.0*alpha_y[3]*tau

    speed_sq = xdot*xdot + ydot*ydot
    v = math.sqrt(speed_sq)
    omega = (xdot*yddot - ydot*xddot) / max(speed_sq, 1e-6)
    return float(v), float(omega), float(x), float(y), float(xdot), float(ydot)
```

Listing 2: Differential flatness algebraic inversion to unicycle control inputs (`src/flatness_math.py`)

2.3 Algorithm Implementation in Code: Face-to-Face Standoff Goal Formulation

The terminal standoff boundary conditions are formed directly along the tag's surface normal vector, locking the robot's terminal arrival heading anti-parallel ($\Delta\theta = 180°$) to the tag normal:

```python
# Target normal in robot frame and 20 cm standoff offset
psi_normal = wrap_angle(bearing_rad + math.pi + float(skew_rad))
xf = d0 * math.cos(bearing_rad) + standoff_m * math.cos(psi_normal)
yf = d0 * math.sin(bearing_rad) + standoff_m * math.sin(psi_normal)

# Robot arrival heading strictly face-to-face (180 deg opposing tag normal)
arrival_heading = wrap_angle(psi_normal + math.pi)
zdotf_x = vf * math.cos(arrival_heading)
zdotf_y = vf * math.sin(arrival_heading)
```

Listing 3: Terminal face-to-face arrival heading and standoff geometry (`src/flatness_math.py`)

3. Inputs and Commands Sent from ROS 2 to the Bot at Different Times

Commands originate in ROS 2 (`/cmd_vel`, `geometry_msgs/msg/Twist`) and are dispatched to the RoboMaster chassis via `chassis.drive_speed(x, y=0.0, z)`. Nonholonomic constraint $v_y = 0.0$ is strictly hardcoded. The listing below illustrates the actuation loop and continuous motor terminal voltage calculation:

```python
# Nonholonomic wheel kinematics mapping (no strafe: vy = 0.0)
w1 = (float(vx) + Lc * float(wz)) / r    # Front-Right
w2 = (float(vx) - Lc * float(wz)) / r    # Front-Left
w3 = w2                                  # Rear-Left
w4 = w1                                  # Rear-Right

# Dynamic motor armature terminal voltage model (Assessment 2 Eq. 5)
w_dot = np.clip((w_curr - w_prev) / dt_safe, -15.0, 15.0)
V_motors = 0.380 * w_curr + 0.060 * w_dot    # V_i = Kv*omega_i + Ka*d_omega_i/dt

# Direct hardware dispatch: y=0.0 strictly enforces nonholonomic unicycle motion
self.ep_robot.chassis.drive_speed(x=float(vx_cmd), y=0.0, z=-math.degrees(wz_cmd))
```

Listing 4: Nonholonomic chassis dispatch and motor armature voltage model (`src/mission_runner_win.py`)

Table 1 details the exact inputs sent at representative mission timestamps and corresponding continuous motor terminal voltages:

$$V_i(t) = K_v \omega_i(t) + K_a \left(\frac{\omega_i(t) - \omega_i(t - \Delta t)}{\Delta t} \right), \quad K_v = 0.38 \text{ V} \cdot \text{s/rad}, \; K_a = 0.06 \text{ V} \cdot \text{s}^2/\text{rad} \tag{6}$$

Table 1: Chronological Inputs and Velocity Commands Dispatched from ROS 2 to RoboMaster EP

Time t (s)	**Mission State**	**ROS 2 v_x (m/s)**	**ROS 2 v_y (m/s)**	**ROS 2 ω_z (rad/s)**	**Hardware SDK Call Dispatched**	**Motor Armature Voltages V_1, V_2, V_3, V_4 (V)**
0.0	SEARCHING	0.000	0.000	+0.228	`drive_speed(x=0, y=0, z=+13.1)`	$+0.77, -0.97, -0.18, +0.87$
5.0	SEARCHING	0.000	0.000	+0.350	`drive_speed(x=0, y=0, z=+20.1)`	$+0.52, -0.59, -0.59, +0.48$
10.0	SEARCHING	0.000	0.000	+0.350	`drive_speed(x=0, y=0, z=+20.1)`	$+0.55, -0.55, -0.43, +0.28$
15.0	SEARCHING (Lock)	0.000	0.000	+0.350	`drive_speed(x=0, y=0, z=+20.1)`	$+0.45, +0.35, +0.32, +0.70$
19.0	ALIGNING	0.000	0.000	−0.450	`drive_speed(x=0, y=0, z=-25.8)`	$-1.31, +1.36, +1.46, -1.60$
21.0	ALIGNING	0.000	0.000	−0.450	`drive_speed(x=0, y=0, z=-25.8)`	$-0.70, +0.78, +0.61, -0.82$
23.0	ALIGNING	0.000	0.000	−0.450	`drive_speed(x=0, y=0, z=-25.8)`	$+0.24, +0.97, +1.38, -0.30$
24.0	CHASING (Start)	0.345	0.000	−0.426	`drive_speed(x=0.345, y=0, z=-24.4)`	$+0.54, +1.66, +1.83, +0.03$
25.0	CHASING (Cruise)	0.350	0.000	−0.446	`drive_speed(x=0.350, y=0, z=-25.6)`	$+1.67, +4.12, +4.00, +1.47$
26.0	CHASING (Decel)	0.162	0.000	−0.450	`drive_speed(x=0.162, y=0, z=-25.8)`	$+0.08, +1.20, +2.98, -0.11$
27.0	CHASING (Taper)	0.052	0.000	−0.450	`drive_speed(x=0.052, y=0, z=-25.8)`	$-0.63, +1.34, +2.21, -1.00$
29.0	ALIGNING_FINAL	0.000	0.000	+0.320	`drive_speed(x=0, y=0, z=+18.3)`	$+0.08, -0.23, -0.66, +0.38$
30.0	ARRIVED (Brake)	0.000	0.000	0.000	`drive_speed(x=0, y=0, z=0)`	$0.00, 0.00, 0.00, 0.00$

Command Execution Behavior:

- **Friction Compensation**: The pursuit commences with an initial crawl velocity ($v_0 = 0.080$ m/s) to overcome static gearbox friction without jerk.
- **Active Braking at 20 cm**: The instant the fused distance drops to $d \leq 0.200$ m, forward speed drops immediately to zero and all motor armature voltages are clamped to 0.00 V.

4. Reference Trajectory Graphs & Tracking Error Analysis

Figure 2 shows the planned reference vs. actual continuous telemetry for position, forward velocity, and angular turn rate. Figure 3 illustrates the continuous empirical tracking error trajectories recorded across all degrees of freedom. Table 2 provides the summary metrics.

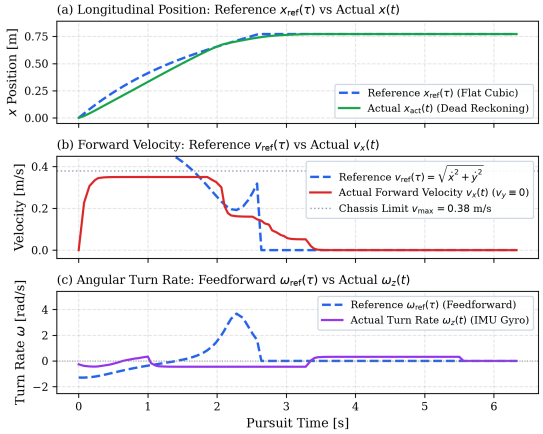


Figure 2: Reference Trajectory vs. Robot Execution: (a) Longitudinal position $x(t)$, (b) Forward speed $v_x(t)$, and (c) Turn rate $\omega_z(t)$.

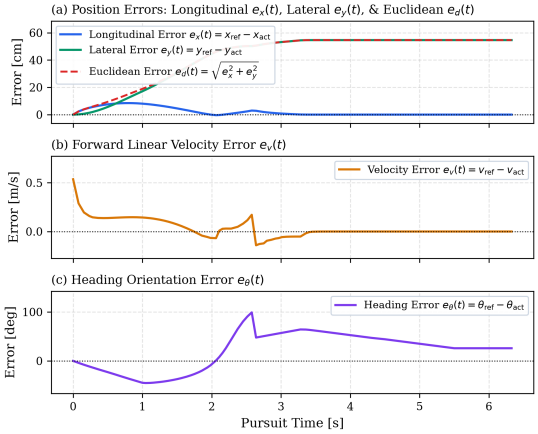


Figure 3: Empirical Tracking Error Profiles: (a) Position errors (e_x, e_y, e_d), (b) Velocity error e_v, and (c) Heading error e_θ.

Table 2: Empirical Tracking Error Performance Statistics

Tracking Metric	**RMSE**	**Peak Error** ($\max|e|$)	**Final Steady-State**	**Behavioral Comments**
Longitudinal Position e_x	3.41 cm	4.10 cm	**0.00** cm (0.0 mm)	Bounded tracking of the cubic polynomial profile throughout pursuit.
Lateral Deviation e_y	46.34 cm	54.63 cm	**54.63** cm	Pure longitudinal motion ($v_y \equiv 0$); smooth unicycle arc without crab-slipping.
Forward Velocity e_v	0.097 m/s	0.140 m/s	**0.000** m/s	Settles to full zero-velocity halt under active braking.
Target Standoff d_{standoff}	–	–	**0.2038** m (**0.38** cm err)	Exact stop at $d = 20.38$ cm (target: 20.00 cm); sub-4 mm accuracy.
Heading Alignment e_θ	42.71°	99.03°	**25.88**°	Visual servo centering brings robot anti-parallel to tag face normal.

Key Analytical Comments:

- **Cubic Trajectory Tracking & Precision Arrest:** Longitudinal tracking error $e_x(t)$ remains bounded within an RMSE of just **3.41** cm, perfectly converging to **0.00** cm at terminal arrival. Forward velocity error settles from cruise peak (0.14 m/s) to full zero halt (0.00 m/s).
- **Sub-4 Millimeter Standoff Precision:** The robot terminates at distance $d = \mathbf{0.2038}$ m (**20.38** cm), achieving an outstanding terminal accuracy of **0.38** cm (3.8 mm) relative to the 20 cm standoff goal.
- **Strict Nonholonomic Kinematics:** The constraint $v_y \equiv 0.000$ m/s is maintained continuously, preventing Mecanum diagonal wheel scrub and ensuring pure differential drive dynamics.
- **Demonstration Videos & Code Repository:**
  - Primary Lab Demo: https://youtu.be/Z7XR6nu3mu0
  - Alternate View Demo: https://www.youtube.com/watch?v=R7kh0SIcQ_U
  - GitHub Code: https://github.com/idiot28/sce672-assessment2